

Reinforcement Learning Framework for Adaptive DBS in the STN-GPe System

1 Goal

The goal is to train an adaptive deep brain stimulation (DBS) controller for the STN-GPe spiking network. The controller should:

- reduce synchrony in the STN network,
- increase spectral entropy of the STN local field potential (LFP),
- use low stimulation energy.

In simple terms, the controller should push the Parkinsonian network away from a highly synchronized, low-entropy state while avoiding unnecessary stimulation.

2 STN-GPe System

The simulator is a two-population spiking model with STN and GPe neurons. The network evolves at a small integration time step:

$$\Delta t = 0.1 \text{ ms.}$$

The original simulator runs a full simulation in one call. For reinforcement learning, we use a steppable wrapper. This wrapper keeps the simulator state in memory and advances the model only for the biological time selected by the agent.

3 MDP Definition

We model the control problem as a Markov decision process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma).$$

At each decision step t , the agent observes a state s_t , chooses an action a_t , receives a reward r_t , and the simulator moves to the next state s_{t+1} :

$$s_t \xrightarrow{a_t} s_{t+1}, \quad r_t = r(s_t, a_t, s_{t+1}).$$

Because the agent chooses the time until the next pulse, one RL step can span a variable amount of biological time. This is closer to a semi-MDP, but we still use the standard Gymnasium interface:

$$(s_{t+1}, r_t, \text{done}, \text{info}) = \text{env.step}(a_t).$$

3.1 State

The environment state is built from recent STN network metrics over a rolling window. The observation is:

$$s_t = \begin{bmatrix} R_t \\ B_t \\ H_t \\ \bar{a}_{t-1} \end{bmatrix}.$$

Here:

- R_t is STN synchrony. Lower is better.
- B_t is beta-band power. It is observed and logged, but it is not used in the current reward by default.
- H_t is spectral entropy of the STN LFP. Higher is better.
- $\bar{a}_{t-1}$ is the previous action, normalized to the range $[0, 1]$. This helps the policy know what stimulation was recently used.

The current metric window is:

$$T_{\text{window}} = 0.25 \text{ s.}$$

With $\Delta t = 0.1 \text{ ms}$, this is:

$$2500 \text{ simulator steps.}$$

3.2 Action

The current action space is continuous. The agent controls:

$$a_t = \begin{bmatrix} A_t \\ T_t \end{bmatrix},$$

where:

- A_t is the DBS pulse amplitude.
- T_t is the pulse period, meaning the time from this pulse to the next pulse.

The current amplitude range is:

$$A_t \in [0, 200].$$

The current frequency range is:

$$f_t \in [5, 50] \text{ Hz.}$$

Therefore the pulse period is bounded by:

$$T_t = \frac{1000}{f_t} \text{ ms,}$$

so:

$$T_t \in [20, 200] \text{ ms.}$$

Each action emits one charge-balanced biphasic pulse:

$$I_{\text{DBS}}(t) = \begin{cases} +A_t, & \text{positive phase,} \\ 0, & \text{interphase gap,} \\ -A_t, & \text{negative phase.} \end{cases}$$

The current phase shape is fixed:

$$\text{phase width} = 0.2 \text{ ms}, \quad \text{interphase gap} = 1.0 \text{ ms}.$$

Only amplitude and pulse period are controlled by the agent at the moment.

3.3 Transition

After the agent chooses a_t , the environment:

1. creates one biphasic DBS pulse using A_t ,
2. inserts that pulse at the start of the decision window,
3. advances the STN-GPe simulator for T_t ms,
4. recomputes the rolling network metrics,
5. returns the next state s_{t+1} .

Mathematically, the transition is:

$$s_{t+1} = P(s_t, a_t),$$

where P is the STN-GPe simulator dynamics plus the DBS input.

4 Reward Function

The current reward is designed to match the project objective directly:

$$r_t = w_H H_t - w_R R_t - \lambda_q q_t - p_t.$$

The current weights are:

$$w_H = 1.0, \quad w_R = 2.0, \quad \lambda_q = 0.005.$$

Here:

- H_t is spectral entropy, so higher entropy increases reward.
- R_t is synchrony, so higher synchrony decreases reward.
- q_t is absolute stimulation charge, so more energy decreases reward.
- p_t is an extra bad-state penalty.

The charge term is computed using absolute current:

$$q_t = \sum_k |I_{\text{DBS}}[k]| \Delta t.$$

Even though the biphasic pulse has net zero charge, we still penalize absolute charge because both phases consume stimulation energy.

4.1 Bad-State Penalty

We add an extra penalty when the network is clearly in a bad state:

$$p_t = \begin{cases} 1.0, & R_t > 0.5 \text{ and } H_t < 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

Therefore the reward becomes:

$$r_t = H_t - 2R_t - 0.005q_t - \mathbb{I}(R_t > 0.5 \text{ and } H_t < 0.5).$$

This means the agent receives a stronger penalty when synchrony is high and entropy is low at the same time.

5 Episode Structure

Each episode has two parts.

5.1 Warmup

The network first runs without stimulation:

$$T_{\text{warmup}} = 0.25 \text{ s.}$$

This lets the network settle and fills the rolling metric buffer. Warmup steps are not counted as RL reward steps.

5.2 Control Period

After warmup, the agent controls the DBS pulses:

$$T_{\text{control}} = 1.0 \text{ s.}$$

Because the agent can choose periods between 20 ms and 200 ms, the number of RL decisions per episode is approximately:

$$5 \leq N_{\text{decisions}} \leq 50.$$

6 Return and Best Model

The return of an episode is the sum of rewards:

$$G = \sum_{t=0}^{T-1} r_t.$$

During training, evaluation is run every fixed number of environment steps. The best model is the one with the highest mean evaluation return:

$$\text{score} = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} G_i.$$

The current evaluation setting uses:

$$N_{\text{eval}} = 3.$$

So the saved best model is not selected from synchrony alone or entropy alone. It is selected using the combined scalar reward, which includes entropy, synchrony, charge, and the bad-state penalty.

7 Algorithms

The code supports three Stable-Baselines3 algorithms.

7.1 SAC

Soft Actor-Critic (SAC) is the current recommended algorithm. It is useful here because:

- the action space is continuous,
- the simulator is expensive,
- SAC is off-policy and can reuse past experience from a replay buffer,
- SAC encourages exploration through an entropy term.

SAC learns a policy:

$$a_t = \pi_{\theta}(s_t),$$

and updates it to maximize expected return while maintaining exploration.

7.2 PPO

Proximal Policy Optimization (PPO) is also supported. PPO is an on-policy method. It is usually stable, but it can require more simulator samples because it does not reuse old data as directly as SAC.

7.3 TD3

Twin Delayed Deep Deterministic Policy Gradient (TD3) is also supported for continuous control. It uses deterministic actions plus exploration noise. It may work, but SAC is usually a better first choice for this system because SAC has stronger built-in exploration.

8 Files in the Codebase

9 Summary

The RL controller observes STN network metrics, chooses adaptive DBS pulse amplitude and timing, and receives reward for lowering synchrony, increasing entropy, and using less stimulation. SAC is the main algorithm because the control problem is continuous and simulator samples are expensive. The best model is saved based on the highest mean evaluation return under the current reward function.

File	Purpose
<code>rl_stn_gpe/env.py</code>	Gymnasium environment for STN-GPe control
<code>rl_stn_gpe/reward.py</code>	Reward calculation
<code>rl_stn_gpe/config.py</code>	Action space, timing, reward, algorithm choice
<code>rl_stn_gpe/stepper.py</code>	Steppable wrapper around the simulator
<code>rl_stn_gpe/train.py</code>	Training script
<code>rl_stn_gpe/eval.py</code>	Evaluation and plotting script
<code>rl_stn_gpe/hparams/sac.py</code>	SAC hyperparameters
<code>rl_stn_gpe/hparams/ppo.py</code>	PPO hyperparameters
<code>rl_stn_gpe/hparams/td3.py</code>	TD3 hyperparameters